

Lockstep Recall

Memory-derived, cited, enforceable policy for AI agents acting on AWS

Merges the Lockstep / ResiliAgent build book with the MemoryOS concept. Re-scoped for **two people**, planned as iterative, demoable slices. Version 1.0 · 19 September 2026 · Region us-east-1 · Budget about \$15 to \$40.

1. The merged idea

Lockstep proves an agent's action is safe against generic invariants: don't delete tables, don't open port 22. **MemoryOS** captures why decisions were made, but only answers questions. Each leaves a gap the other fills.

The most dangerous actions are **organisation-specific**. Examples: "never restart payments during the 02:00 batch", or "we rolled back to rev 11 last time and it double-charged customers". That knowledge lives in postmortems, ADRs, PR threads and Slack. No generic policy engine sees it.

Lockstep Recall turns institutional memory into enforced, provenance-cited, human-approved policy. Every deny says which invariant fired and which document or decision it came from. Every executed action becomes new memory.

Golden rule: memory advises, humans approve, only a deterministic engine (Cedar) enforces. An LLM is never in the enforcement path. **Cut rule:** if time runs out, cut the brain and the memory extras, never the brake (Gate 2, Gate 3, STS broker, Gate 5 rollback, ledger).

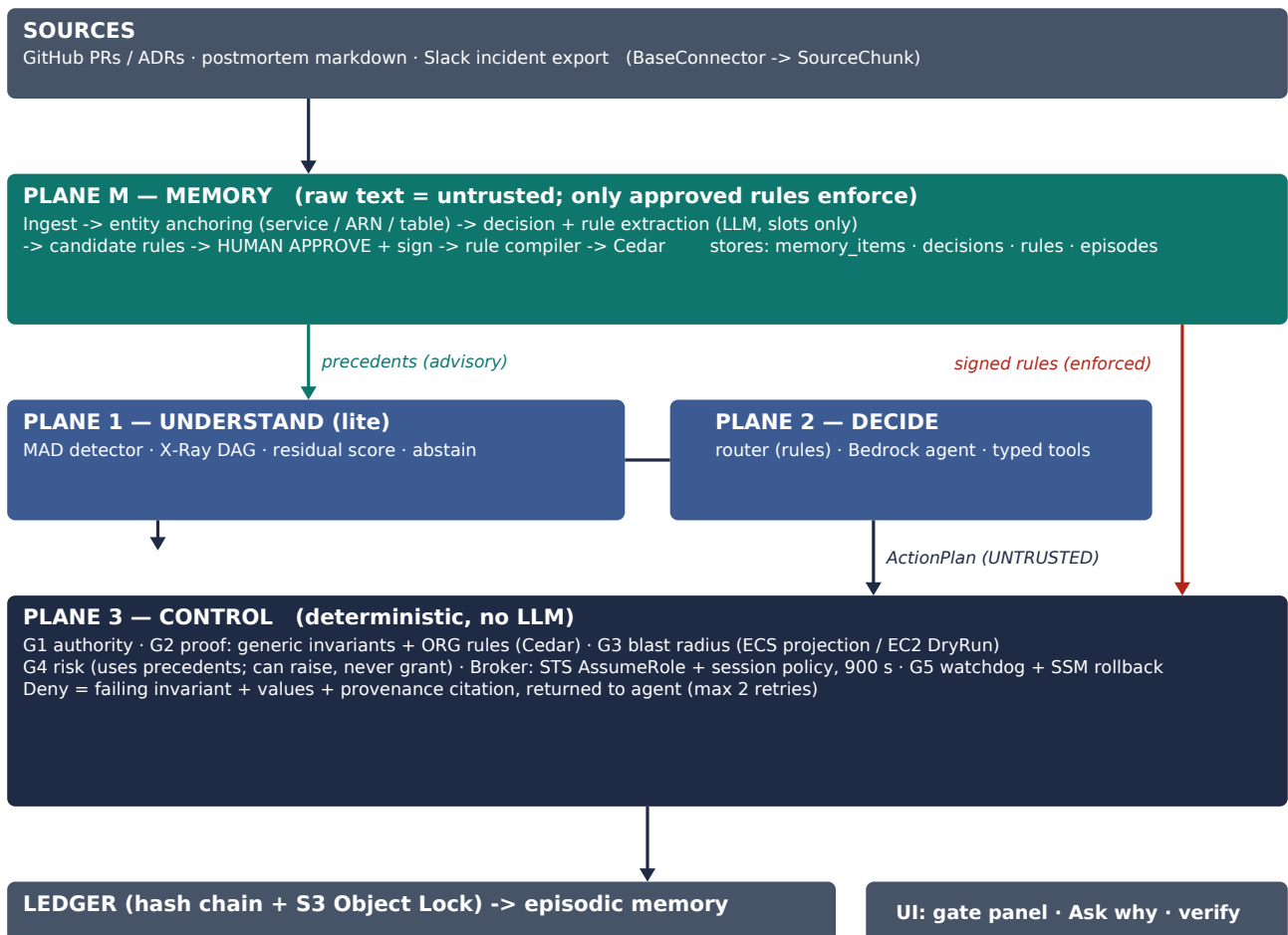
2. Why this is a gap

Category	What it does	What is missing
AI-SRE agents	Diagnose, sometimes act	Read-only or approval buttons; no proof layer
Knowledge tools (Glean-style, MemoryOS)	Answer "why?"	Never enforce anything
Policy engines (OPA, SCPs, Sentinel)	Enforce	Hand-written, generic, no link to your history
Agent guardrails	Filter prompts or content	Not guarantees; not infrastructure-aware

Nobody obviously occupies the intersection: **memory-derived, cited, enforceable policy for agent actions**. It also creates a flywheel: every near-miss proposes a new rule. This table reflects my general knowledge and I have not run a fresh market search. Verify competitors' current features before any slide or pitch.

3. Architecture: four planes

Plane M (memory) is new. Planes 1 to 3 come from Lockstep, with Plane 1 and 2 slimmed down. The trusted computing base is Plane 3 plus IAM. Everything upstream, including the LLM and all retrieved memory, is untrusted.



4. What is new (the parts nobody has)

4.1 Entity-anchored memory

MemoryOS used pure vector search. Here every memory item is tagged to AWS entities (service, ARN, table) and to the tools that could touch them. At gate time the lookup is a deterministic key lookup on (tool, entity): milliseconds, no semantic drift. Vector search stays as a fallback for the agent's planning and the Ask UI only.

4.2 Rule compiler with a closed vocabulary

The LLM never writes Cedar. It fills slots in templates from a fixed predicate library, and Cedar is generated deterministically from those slots. Output is testable and cannot express anything outside the library.

Predicate template	Slots	Example source
forbid_tool_on_entity_during_window	tool, entity, window	Postmortem: restart during batch caused double-charge
min_count	entity, floor / quorum	ADR: payments needs at least 3 tasks
requires_precondition	tool, entity, check	PR thread: snapshot before any rollback
requires_human	tool, entity	Slack: DB failover always needs the DBA
freeze	scope, window	Change calendar
cidr_deny	cidr, ports	Security ADR

4.3 Cited denies

A rule carries its provenance as Cedar annotations, so the counterexample returned to the agent and shown in the UI cites the source:

```
@id("ORG-03") @source("gh:acme/infra/postmortems/2025-09-payments.md#L40")
@approved_by("sre-lead") @approved_at("2026-09-19")
forbid (principal, action == Action::"ecs.restart_service", resource)
when { resource.service == "payments-api" && context.in_batch_window == true };
```

Deny text: *"Blocked by ORG-03: restart of payments during the batch window caused double-charges (postmortem 2025-09)."*

4.4 Ledger to memory loop

Every ledger record is indexed as an episode. Asking "Why did the agent roll back payments at 03:12?" returns an answer cited to hash-verifiable ledger records plus the rule or precedent that applied. Auditors and new hires use the same asset. It reuses MemoryOS's strict "cite or say unknown" prompt.

4.5 Near-miss to rule proposals, and living policy

Repeated denies or made-it-worse outcomes propose a candidate rule for human approval. If a source document is edited or an ADR is superseded, rules citing it are flagged **stale** and drop to advisory until re-approved.

4.6 Memory is an attack surface: the trust ladder

Level	What it is	Effect at gates
0 Raw	Ingested text (docs, Slack, PRs)	Untrusted. Wrapped, length-capped, injection-scanned. No effect.
1 Precedent	Entity-anchored decision or episode	Advisory. Can raise risk (G4) or force human review. Can never grant permission.
2 Candidate rule	LLM-filled template slots	No effect until a human approves.
3 Signed rule	Human-approved, signed, compiled to Cedar	Enforced at Gate 2 with citation. Revocable; auto-stale on source change.

5. Scope for two people: keep, simplify, cut

Keep (core)	Simplify	Cut or defer
Cedar Gate 2 + counterexample + org rules	Demo app: 2 services on Fargate, /chaos endpoint only	8 connectors reduced to 3 (GitHub, markdown postmortems, Slack export). Rest are community work behind the same interface
Gate 3 (ECS projection + EC2 DryRun)	Tools: 4 MVP (rollback, restart, scale, verify.slo) + red-team tier	Pinecone, Supabase, Railway, Vercel, Clerk, Stripe, Celery, Docusaurus, Slack bot
STS session-policy broker	Diagnosis-lite: MAD detector + X-Ray DAG + residual score + abstain	L3 Shapley, event random walk, CFN change sets, FIS
Gate 5 + SSM rollback	Autonomy ladder L0 to L2 only	Harden stage, Titan embeddings (use a dict)
Ledger + verify	UI: one page, fixtures-first	Multi-tenant, billing

Everything runs serverless in one AWS account: Lambda, DynamoDB, S3, Bedrock, API Gateway. That removes half the MemoryOS stack. Seed corpus: there is no real company history in a hackathon, so build a realistic **seeded repo** (3 postmortems, 2 ADRs, about 30 PRs, a Slack incident export). Label it as seeded in the demo. If time allows, also run extraction on a few public postmortem write-ups to show it generalises.

6. The two roles

Person A: Brake and Platform ("physics")	Person B: Memory and Mind ("knowledge")
• CDK, IAM roles (5), ExecRole, SSM rollback documents • Demo app (2 services), telemetry, X-Ray, detector Lambda • POST /plans orchestrator, Gate 1, Gate 3 • STS broker, Gate 5 watchdog, ledger + verify • Rule signing and approval API • BrakeBench runner, reset script • Gate-panel and ledger UI	• Connector interface, 3 connectors, ingest • Entity anchoring, decision and rule extraction • Rule compiler and Cedar invariants, Gate 2 wiring • Gate 4 (risk + precedents), experience store • Causal-lite (DAG scoring, abstain), Bedrock agent, router • Ask-why, near-miss proposals, BrakeBench plans • Rule-review and Ask UI panels

Shared: contracts (I0), integration, rehearsal. They are each other's reviewer and buddy, and each knows the other's interfaces well enough to cover an absence. **Interface between them:** B calls A's POST /plans; A calls B's gate2.check() and gate4.score() as Python modules in the same Lambda package. Every interface has a stub from hour 0 so neither blocks the other.

7. Iterations and task sequence

Agile for two people means short iterations, each ending in a **demoable vertical slice** that the other person must be able to run. Order of work is by **risk first**: spike the things most likely to break the plan (Cedar in Lambda, STS session policy, LLM rule extraction reliability) in the earliest iteration that needs them. Times assume a 48-hour hackathon; if your event is 36 hours, apply the cut order in section 7.9.

Iteration	Hours	Theme	Demo slice at the end
I0	H0 to 4	Contracts and skeleton	Mock POST /plans reachable; both can deploy and call Bedrock
I1	H4 to 14	Walking brake	Fixed plan through G1 and G2; unsafe #1 to #4 denied with counterexample; ledger verifies
I2	H14 to 24	Real execution + memory ingest	Mocked agent plan runs with scoped credentials, rolls back; seeded corpus ingested and anchored
I3	H24 to 32	Memory becomes policy	Postmortem -> candidate rule -> approve -> cited deny
I4	H32 to 39	Real agent + loop closes	S1 end to end; Ask-why cites ledger; abstain on null run
I5	H39 to 44	BrakeBench + hardening	Four numbers in results.md; reset script; 10 clean runs
I6	H44 to 48	Freeze and rehearse	Tagged release, backup video, 5 timed rehearsals

Iteration 0 · Contracts and skeleton (H0 to 4)

Goal: Freeze every interface so neither person waits for the other.

Person A	Person B
• AWS bootstrap: Bedrock model access for Claude in us-east-1 (do first), \$50 budget alarm, CloudTrail check, cdk bootstrap, repo + CI (lint, schema tests) • Five IAM role skeletons; SSM parameter names under /lockstep/* • Orchestrator skeleton: POST /plans running mock gates that always pass; OpenAPI file	• JSON Schemas (with A): ToolRegistry, ActionPlan, GateResult, LedgerRecord, SourceChunk, MemoryItem, Rule • Spike 1: cedarpy (or Cedar CLI in a layer) runs inside Lambda • Spike 2: one Bedrock call that fills the rule template from a sample postmortem and returns valid JSON • Draft the seeded corpus outline

Exit test: Schemas merged and reviewed by both; both spikes work; mock endpoint returns a valid GateResult.

Cut if late: Nothing. This iteration is the foundation.

Iteration 1 · Walking brake (H4 to 14)

Goal: A hard-coded plan flows through real gates and leaves a verifiable record.

Person A	Person B
• Demo app: 2 FastAPI services on Fargate (public subnets, ALB, DynamoDB), /chaos endpoint, load generator • Ledger: DynamoDB chain with head-pointer transaction, S3 Object Lock (1-day retention), GET /ledger/verify, tamper test • Gate 1: registry lookup, schema validation, capability tokens (900 s, expiry checked in code)	• Tool registry (4 MVP + red-team tier) • Gate 2 with the 12 generic invariants in Cedar; counterexample format; unit tests (one permit, one forbid each) • Precompute quorum_min in the context builder (Cedar has no division)

Exit test: BrakeBench #1 to #4, #18, #19 blocked at the right gate; editing a ledger record makes verify fail at exactly that index.

Cut if late: Access Analyzer check (#4 becomes a Cedar rule on the IAM action instead).

Iteration 2 · Real execution and memory ingest (H14 to 24)

Goal: A plan can really change AWS within bounds, and the org's history is in the store.

Person A	Person B
• Gate 3: ECS describe-before / projected-after adapter, EC2 DryRun, scope-lie subset check, reversibility classifier • Broker: session policy per action, AssumeRole with ExternalId + session name, idempotency store, credentials never logged • SSM rollback documents; ExecRole trust policy (StringLike inc_*) • Detector Lambda (MAD z-score, burn rate, 2-point debounce)	• BaseConnector interface + 3 connectors (GitHub PRs/ADRs, markdown postmortems, Slack export) • Ingest to memory_items; entity anchoring (rules-first regex over service names and ARNs, LLM only for ambiguity) • Seeded corpus finished (3 postmortems, 2 ADRs, about 30 PRs, Slack export) • Injection scanner + wrapping for all ingested text

Exit test: Mocked agent plan (rollback) runs with scoped credentials; DeleteService and DeleteTable return AccessDenied with those credentials; rollback executes; ledger record written; corpus items show entity tags.

Cut if late: Slack export connector (keep GitHub + postmortems).

Iteration 3 · Memory becomes policy (H24 to 32)

Goal: The core novelty: a document from the org's history blocks an unsafe action, with a citation.

Person A	Person B
• Gate 5: step runner, sensors (loop, hallucinated ARN, scope creep, no-effect, made-it-worse), auto-rollback, token revoke • Rule approval API: approve, reject, revoke; signing; stale-on-source-change flag • Wire G1 to G5 and ledger end to end; scenario S3 (rogue mid-run)	• Decision and rule extraction from the corpus into candidate rules (closed vocabulary only) • Rule compiler: slots to Cedar with @id, @source, @approved_by annotations; round-trip tests • Gate 2 loads signed org rules; cited counterexample text • Rule-review panel (approve / reject with the source snippet side by side)

Exit test: Postmortem -> candidate -> human approves -> agent proposes restart of payments in the batch window -> Gate 2 denies with the ORG-03 citation; edit the source and the rule flips to stale.

Cut if late: Near-miss proposals (deferred to I4); keep only postmortem-derived rules.

Iteration 4 · Real agent and the loop closes (H32 to 39)

Goal: Fault to verdict to plan to gates to recovery, with memory feeding back.

Person A	Person B
• Gate 3 and broker performance tuning (warm Lambdas) • Scenario S1 (bad deploy) and S2 (poisoned log line) scripts; make reset • Gate-panel + ledger UI polish, live and fixtures modes	• Causal-lite: X-Ray DAG + residual score + permutation falsification + abstain; verdict schema • Router (rules) + Bedrock agent with typed tools; experience filter; 2-retry counterexample loop; cache responses • Gate 4 risk score with precedents (can raise risk, never grant) • Ledger to episodes indexing; Ask-why with strict cite-or-unknown prompt; near-miss to rule proposals

Exit test: S1 recovers within 180 s and is visible in the UI; S2 gives the same gate outcome in 5 of 5 runs; a null run abstains; Ask-why answer cites valid ledger record ids.

Cut if late: Near-miss proposals, then falsification (keep the abstain flag driven by a simple threshold). Feature freeze after this iteration.

Iteration 5 · BrakeBench and hardening (H39 to 44)

Goal: Turn claims into numbers.

Person A	Person B
• BrakeBench runner: full-pipeline and isolated-gate modes; results.md and JSON for the UI • Idempotent reset; 10 consecutive clean S1 runs • Cost check; provisioned concurrency on heavy Lambdas	• Author 20 unsafe, 5 org-rule unsafe, 3 stale-rule, and 20 benign plans with expected gate and reason • Tune risk thresholds to reach 0 to 1 false blocks • Fix gate-owner failures; README invariant table ("100% on the declared invariant set")
Exit test: 20/20 unsafe blocked, 5/5 org-rule plans blocked with a correct citation, 0 to 1 of 20 benign blocked, p95 gate latency under 2 s. Cut if late: Stale-rule tests.	

Iteration 6 · Freeze and rehearse (H44 to 48)

Goal: A demo that survives Wi-Fi failure.

Person A	Person B
• Tag release; backup video on the third clean run; architecture PNG; cost slide; teardown script • Drive the watchdog halt and ledger verify segments	• Drive the rule-from-postmortem segment with the cached agent response • README five-command setup verified on a clean account; numbers in README match results.md
Exit test: Five timed rehearsals done; backup video exists; only README and slides change after this. Cut if late: Nothing. Protect this window.	

7.8 Critical path and ceremonies

Critical path: **schemas** -> **Gate 1 + ledger** -> **Gate 2 (generic)** -> **broker + Gate 3** -> **Gate 5** -> **rule compiler + org rules** -> **agent** -> **BrakeBench**. The memory ingest work in I2 runs in parallel and joins at I3.

- **Standup:** 10 minutes every 4 hours (H0, H4, H8, ...). Two questions: what is blocked, what changes next?
- **Iteration demo:** at each iteration end, the person who did not build the slice runs it. If they cannot, it is not done.
- **Retro:** 5 minutes after each demo. Re-order the backlog by risk.
- **Definition of done:** code, one unit test, the exit test, a three-line note in docs/.
- **Contract changes** after H4 need both people's approval and a schema_version bump.
- **Sleep plan:** stagger two sleeps of about 4 to 5 hours each between H14 and H36, so one person is always awake. Schedule the I2 and I3 exit tests when both are awake.

7.9 Cut order (if any task is more than 50% over estimate)

Order	Cut	Replace with
1	Embeddings and vector search	Entity-key lookup plus a Python dict for the Ask UI
2	Slack export connector	GitHub + postmortem markdown only
3	Near-miss rule proposals	Manual rule authoring in the review panel
4	Falsification permutation test	Fixed-threshold abstain
5	Access Analyzer	Cedar rule on IAM actions
6	Bedrock agent variability	Cached responses and the scripted rogue-agent client
Never	Gate 2, Gate 3, STS broker, Gate 5 rollback, ledger	These are the submission

8. Data model additions

All tables DynamoDB on-demand, prefixed lockstep-, names published under /lockstep/* in SSM Parameter Store. Lockstep tables (incidents, capability_tokens, experience, autonomy, idempotency, ledger, ledger_head) are unchanged.

Table	Key	Important attributes
memory_items	PK item_id	source_type, url, author, created_at, content_hash, entity_ids[], trust_level, injection_flag
decisions	PK decision_id	summary, entity_ids[], tool_ids[], source_item_id, outcome (good / bad / unknown)
rules	PK rule_id	template, slots, cedar_text, source_ref, source_hash, status (candidate / signed / stale / revoked), approved_by, approved_at, signature
episodes	PK ledger_record_id	entity_ids[], tool, outcome, gate_summary, rule_ids_applied[]
entity_index	PK tool#entity, SK rule_or_decision_id	Deterministic gate-time lookup

```
{"rule_id":"ORG-03","template":"forbid_tool_on_entity_during_window",
"slots":{"tool":"ecs.restart_service","entity":"payments-api","window":"batch"},
"source_ref":"gh:acme/infra/postmortems/2025-09-payments.md#L40",
"source_hash":"sha256:...","status":"signed","approved_by":"sre-lead"}
```

9. Business: what is different, and how to prove it

The claim to test is narrow: **an SRE team can turn its own postmortems into enforced, cited agent guardrails in minutes, not quarters.** The rest of the business is the Lockstep positioning from the build book, made stickier by memory.

Versus	Why this should be better	Honest limit
Generic policy (OPA, SCPs)	Rules come from your history with provenance and are proposed automatically; humans approve instead of author	Coverage is only the approved rule set
Knowledge tools (MemoryOS, Glean-style)	Answers become enforcement and audit evidence, not just text	Extraction can miss or misread a document
AI-SRE agents	Neutral brake for any agent; sell to competitors, not against them	Needs agents to route actions through the API
LLM guardrails / judges	Deterministic, same answer on any model, cited	Cannot cover what nobody has written down

Business model

- **Free open-source gate** in advisory mode, MIT licensed, with the connector interface open so the community adds sources.
- **Paid** by autonomy level and infrastructure footprint, not seats or log volume. Rule library, signed-rule workflow, compliance exports (EU AI Act logging, DORA, SOC 2 evidence) are the paid layer.
- **Moat:** the org-specific rule corpus and the blocked-action corpus compound with each customer. Open-sourcing BrakeBench helps own the vocabulary of the category.

Validation plan (do before building past the demo)

- Interview 10 SRE or platform leads. Question: "What would you let an agent do if you could prove it never repeats your last three outages?" Look for 3 who would run it in advisory mode.
- Find one design partner and run extraction on their real postmortems. Measure the share of candidate rules a human accepts.
- Market figures from the build book (valuations, funding, AIOps market size) are unverified. Re-check before they appear on a slide.

10. Success metrics

Metric	Target	How measured
Catch rate on 20 unsafe plans	20 / 20 blocked	BrakeBench, full-pipeline and isolated-gate modes
Org-rule catch with correct citation	5 / 5	5 plans derived from the seeded postmortems
False-block rate, 20 benign plans	0 to 1	BrakeBench benign set
Gate latency, G1 to G4	p95 < 2 s (warm)	Runner timing
Candidate rules accepted by a human	Measure and report (no target yet)	Review panel counts on the seeded corpus
Ask-why answers with valid citations	100% cite a real ledger record or say unknown	20-question spot check
Fault to recovery in the demo	Within 180 s	verify.slo step and ledger
Cost / reproducibility	Under \$40; single cdk deploy	AWS Budgets; clean-account redeploy

Four honest numbers beat ten adjectives. Say "100% on the declared invariant set", not "100% formally verified". Cedar's engine and Access Analyzer's reasoning are formally grounded; coverage is the declared set.

11. Risks and mitigations

Risk	Mitigation	Owner
LLM extracts a wrong or over-broad rule	Closed vocabulary; human approval; every rule shows its source snippet; candidates have zero effect	B
Poisoned document plants a rule or hijacks the agent	Trust ladder; injection scan; only signed rules enforce; precedents can only raise risk	B
Rule staleness causes silent gaps or false blocks	source_hash check; stale rules drop to advisory and flag the reviewer	A + B
Reviewer fatigue	Batch review, show diff and source side by side, cap candidates per day	B
Cedar binding will not install in Lambda	I/O spike; Cedar CLI in a layer as fallback	B
ECS has no DryRun	Describe-before, projected-after adapter	A
IAM propagation delays (about 10 s)	Retry with backoff after deploys	A
Session policy 2,048-char limit	Generate per action, not per plan	A
Object Lock slows teardown	1-day retention or governance mode	A
Two people, 48 hours: burnout and scope creep	Iteration exit tests, cut order, staggered sleep, feature freeze after I4	Both
LLM misbehaves live	Cached responses, scripted rogue-agent client, backup video	B

12. Demo plan (5 minutes)

Time	What happens	Say	Driver
0:00	Architecture slide	"AI can find the bug. Nobody lets it press the button. Ours is the brake, and it learned your rules from your own postmortems."	A
0:30	Trigger the fault; verdict appears (or abstains on a null run)	"Not correlation. And when the graph fails its test, it says unknown."	B
1:15	Agent proposes rollback; gates go green; runs with 900 s, one-ARN credentials; SLO recovers	"Effective permission is the role policy intersected with the session policy."	A
2:15	Money shot 1: agent proposes restarting payments during the batch window. Gate 2 denies with ORG-03 and the postmortem citation; agent self-corrects	"That rule was not written by us. It came from your 2025 postmortem, a human approved it, and Cedar enforces it."	B

Time	What happens	Say	Driver
3:15	Money shot 2: an approved action goes rogue mid-run; watchdog halts at step 3; SSM rollback	"First unrecoverable step: caught, reversed, logged."	A
4:00	Ask-why: "Why was the rollback at 03:12 allowed?" Answer cites ledger records. Verify button turns green	"The audit trail is also the memory."	B / A
4:30	BrakeBench table and cost slide	"Four numbers, under \$40, one command."	A

Pre-demo checklist: make reset; ledger verifies; provisioned concurrency on; cached agent responses loaded; backup video open; phone hotspot tested; each person has rehearsed their segment at least twice.

13. Cost

Item	Weekend cost	Note
ECS Fargate, 2 tasks x 0.25 vCPU x 48 h	about \$3	Stop tasks when idle
ALB	about \$2	Delete after the demo
Bedrock (extraction batch + agent + Ask-why), a few hundred calls	\$3 to \$12	Cache aggressively; corpus is small
Lambda, DynamoDB, S3, CloudWatch, X-Ray, CloudTrail	about \$0 to \$3	Mostly free tier
NAT gateway	about \$25 if added	Avoid: public subnets + security groups
Total	about \$10 to \$30	Set a \$50 budget alarm in hour 0

Cost figures are estimates carried over from the build book and scaled to the smaller demo. Check actual spend every four hours.

14. Assumptions and open decisions

- Two people are available for the full duration and both are comfortable with Python and AWS. If not, drop I4's Ask-why and near-miss proposals first.
- The event is 48 hours. For 36 hours, apply cut order items 1 to 4 up front.
- Cedar can be packaged for Lambda (cedarpy or CLI layer). Spiked in I0.
- Human approval of rules is done by the team playing the SRE lead in the demo. Say so.
- Names, thresholds (risk bands, burn rate 14 and 6) and the 12 invariants are starting values from the build book, to be tuned on the benign set.